Department of Computer Science and Engineering
Tandon School of Engineering
New York University
Professor: Gustavo Sandoval

Homework 1 : sed

CS-UY 3224, Intro to Operating Systems

Abstract

This homework is designed to ensure you are familiar with the tools and programming skills necessary to succeed in this course. In it you will set up an integrated development environment (IDE) as necessary to work on the pedagogical operating system we teach with, run that operating system on a machine emulator, and write some simple user programs in the C programming language.

Academic Integrity

All work submitted for this assignment must be your own. Cheating and plagiarism will not be tolerated. NYU Tandon's policy on academic misconduct[1] applies to this assignment. If you have questions about appropriate conduct with regards to homework that are not answered by the syllabus or Tandon's code of conduct, please contact your professor.

Part 1: Setting Up Anubis

Anubis is a cloud IDE and assignment submission service that we use for this course. *All students*, will use Anubis for assignment submission. The course also optionally requires a collection of tools, namely:

- gcc-10, or later
- binutils, with i386 support
- qemu-system-i386
- git
- make

If you have all of these tools and a local development environment, you are free to use them. If you do not, the Anubis cloud IDE environment provides everything already setup for you.

To access the assignment in Anubis:

1. Navigate to `https://anubis-lms.io`
2. Sign in with your NYU ID using the "Sign In" button at the top right of the page
3. Navigate to your profile icon at the top right of the page, click "View Profile" in the drop down
4. Link your Anubis account with your Github account using the "Link Account with Github" button
5. Navigate to "Courses" in the left sidebar and click "Join Another Course".
   The course code is **iloveos**.
6. Navigate to "Assignments" and "Homework 1: sed"

[1] NYU Tandon Student Code of Conduct

7. Click "Create Repo"

At this point you may navigate to the repo with the "View Repo" button. You are free to work on this repository using your own workflow. Every commit in the "main" branch at the close of the submission period will be considered a "submission" for the purposes of grading. The latest or most complete commit will be the one that is graded.

If you choose to use the Anubis Cloud IDE, Anubis will auto-commit your work every few minutes if you leave the "Autosave Enabled" switch on. You can manually commit your work with the `anubis autosave` command in terminal.

Some important commands to get you started:

- `make`: builds the operating system kernel and user programs
- `make clean`: removes all compiled files from the source tree
- `make qemu`: builds the operating system and launches the emulator
- When inside the emulator, "Ctrl + a" followed by "x" will exit the emulator

Part 2: Hello World

The goal of this part is to write your first xv6 user program, "Hello World".

1. Create a new file, "hello.c", in the "user" folder
2. Add "`$U/_hello\`" to "`UPROGS`" in the "Makefile"
3. Populate "hello.c" with code to print "Hello World." exactly followed by a newline character, and then exit without error

Hint: Look at other user progams like wc.c, ls.c, and cat.c to get an idea of how to implement hello.

Part 3: sed

The goal of this part is to familiarize yourself with C and Unix programming concepts such as program arguments, pipes, file descriptors, I/O, memory allocation, functions, and pointers.

1. Create a user program, `sed`, in the same manner as the `hello` program from Part 2
2. The program should read text from a file or standard input, perform a word replacement, and output the result to standard output
3. The program will always be called with at least two arguments in the format: `sed -OldWord -NewWord`, where `OldWord` is the word to be replaced and `NewWord` is the word to be inserted
4. The program will optionally have a third argument, a file name, to read input text from. If no file name is provided, the program should read from standard input
5. Your program must work on arbitrarily large input amounts of input text, and the `OldWord` and `NewWord` may be of any size, and may be different sizes than one another
6. Your program should not invoke unsafe or undefined behavior. Additionally, it should contain sufficient comments and use appropriately concise functions to be understandable to graders

Hints:

- xv6 does not provide the C standard library. The functions available to user programs in xv6 are listed in the "user.h" header.
- You can add a test file with numbers to the xv6 file system by adding it to the `fs.img` target in the "Makefile". For example, if you had a file called "input.txt" you would change the `mkfs` call to `./mkfs/mkfs fs.img README.md input.txt $(UPROGS)`
- Some various tests you might be interested in trying:
  - `sed -Old -New input.txt`
  - `cat input.txt | sed -Old -New`
- Assuming "input.txt" contains the string "abc123" the command: `sed -bc -efg input.txt` should result in an output of "aefg123"
- Assuming "input.txt" contains the string "the quick brown fox jumped over the lazy dog" the following results should occur:
  - `sed -the -xyz input.txt` → "xyz quick brown fox jumped over xyz lazy dog"
  - `sed -jumped -fell input.txt` → "the quick brown fox fell over the lazy dog"

Grading

Part 1 & 2	40 points

Part 3

Reads from file	10 points
Reads from stdin	10 points
Handles arbitrary text sizes	10 points
Replacement works	10 points
Does not introduce extraneous output	10 points
Safety, Comments, Style	10 points

Various functions of your program may not be completely covered by this rubric, bugs and other non-functional elements should be documented in your comments. A documented bug with a note of what was done to try to fix it will be penalized less than bugs discovered during grading.